FHF SALUD

TAR05 - Equipo Amarillo

Ingeniería de Software (SCD-1011)
Instituto Tecnológico Superior de Xalapa

Auditoría, integración y consolidación de base de datos

6 de noviembre de 2026

Estado: validación local aprobada.
Aplicación y validación en Supabase remoto bloqueadas.

Autores verificables en Git: Rodrigo Ruiz Aguilar; Emilio Martinez; yaird meyer; jesus_ff; Manuel Alvarez.

No se proporcionó un reporte TAR05 previo con portada o firmas. No se generaron firmas ni se completaron identidades. El PDF PRY05 original se conserva intacto.

Ingeniería de Software SCD-1011. Instituto Tecnológico Superior de Xalapa. Fecha de auditoría: 6 de noviembre de 2026. Estado: correcciones y pruebas locales verificadas; entrega incompleta por bloqueo de Supabase remoto.

A. Resumen ejecutivo

Se actualizó la copia desde 5bc865ba61bbad849370aa4091ea30b38c62fd32 hasta e5299b5144d84723bd9ab77e904a54a72f4efc39 mediante fast-forward. Se preservaron los cambios locales antes de sincronizar. Se verificaron las cinco áreas PRY05 por código, historial y pruebas. La integración remota incluía un esquema alternativo incompatible que impedía ejecutar las migraciones secuencialmente. Además, las RPC SECURITY DEFINER omitían el rechazo de una cuenta autenticada sin rol debido a la lógica ternaria SQL. Ambos problemas se reprodujeron y corrigieron.

El modelo consolidado dispone de cinco tablas funcionales normalizadas. Las migraciones de citas y las pruebas se ejecutaron sobre PostgreSQL 17 real, aislado, sin red, con un esquema auth mínimo para simular claims exclusivamente en pruebas. Esto NO equivale a un proyecto Supabase remoto, ni valida el proveedor Auth o sus JWT. No se modificó la configuración privada existente ni se introdujeron datos simulados en producción.

La instalación y npm run verify aprobaron lint estricto, tipos, 75 pruebas Jest, 8 pruebas Python, 4 pruebas Node y build. La UI real sin sesión se comprobó con Playwright en escritorio, tablet y móvil. El recorrido autenticado de navegador contra Supabase sigue pendiente; las capturas PRY05 son evidencia histórica, no una ejecución nueva TAR05.

B. Git y colaboraciones verificables

Repositorio oficial: https://github.com/Vo1dScripter/FHF-Salud. origin conservado: git@github.com:Rodri040409/FHF-Salud.git. Se añadió oficial sin cambiar origin. Ambos main remotos convergieron en e5299b5144d84723bd9ab77e904a54a72f4efc39. Rama de trabajo: feature/tar05-consolidacion-bd. No había commits locales inéditos; sí documentación modificada y archivos sin seguimiento.

Respaldo externo: /home/voidscripter/.codex/tmp/fhf-tar05-backup-RHDBf8 (patch binario y tar de archivos sin seguimiento). Respaldo Git: stash creado el 6 de noviembre, mensaje “respaldo previo TAR05 2026-11-06”. Se conserva sin aplicar para no mezclar documentación antigua con la integración nueva. No se ejecutaron reset --hard ni clean. No se borraron ramas ni se reescribió historial.

Integración 1: c2a2e5d099e8dee25d67c70aaf4a640fe7f8ebed (c2a2e5d), Rodrigo Ruiz Aguilar, 2026-10-25T19:35:20-06:00. Mensaje real: feat: preparar citas con temas claro/oscuro e integrar Supabase. No existe el mensaje exacto chore solicitado. Aporta shell, rutas, contrato, temas, cliente Supabase y preparación del módulo. Integrado en main. No se puede demostrar la rama original solo por el grafo actual.

Integración 2: 7963f97d4e041be1ea183152e3423f147d410b04 (7963f97), Emilio Martinez, 2026-10-25T22:04:49-06:00. feat: modelar citas y disponibilidad. Rama feature/base-datos-citas verificable por la referencia remota 698f436 y el merge e5299b5. El esquema alternativo duplicaba tablas, utilizaba scheduled_at en lugar de starts_at y políticas de pacientes incompatibles con la app. Fue integrado, aunque la documentación PRY05 decía que no. Se conserva la propuesta original en docs/sql-historico; el número de migración ahora comprueba el contrato canónico sin crear tablas duplicadas. No se afirma que el remoto haya aplicado ninguno de los dos esquemas.

Integración 3: 9b8ee79593b02d451602004049f00d006f4b2031 (9b8ee79), yaird meyer, 2026-10-26T00:06:28-06:00. feat: registrar consultar y validar citas. API y servicio RF-01/RF-02/RF-05, adaptador PostgREST real, validación y pruebas. Integrado. No usa arrays locales como persistencia de

producción; los dobles en tests no son la fuente de datos de la app. Rama original no demostrable.

Integración 4: 77bd0310762b91b0aa143356ed6721f05081193f (77bd031), jesus_ff, 2026-10-26T01:32:49-06:00. feat:gestionar estados y cambios de citas. RPC con bloqueo de fila, versión esperada, historial y diálogos de acciones RF-03/RF-04/RF-06. Integrado. TAR05 corrige la autorización NULL de las RPC por migración posterior. Rama original no demostrable.

Integración 5: ef8a8bf4f86e1f857f260a02ab120fe410caf26a (ef8a8bf), Manuel Alvarez, 2026-10-30T15:08:38-06:00. fix: integrar módulo completo de citas. QA, documentación y evidencias históricas. Merge final e5299b5 por Manuel Alvarez, 2026-10-30T15:37:35-06:00, incorpora feature/base-datos-citas. Los nombres son exactamente los de Git, no identidades legales inferidas. No se generaron firmas. Archivos modificados por cada commit: historial-git.txt; ramas: ramas.txt. El commit final TAR05 se obtiene con git log -1 --format=fuller en esta rama; su hash se entrega en el cierre de ejecución.

Stack real y justificación

Confirmados en package-lock: Next.js 16.2.0, React 19.2.4, TypeScript 5.9.3, @supabase/ssr 0.12.7, @supabase/supabase-js 2.118.0, Tailwind CSS 4.2.2 y Jest 30.3.0. Node local v22.23.1. npm y package-lock son el gestor y lock efectivos. SCSS/Gulp compila los estilos existentes; no se rediseñó la aplicación. App Router y rutas API comparten contratos con features/appointments. Zod está disponible; no debe confundirse una dependencia instalada con uso en cada validador.

PostgreSQL/Supabase es la persistencia implementada por el adaptador; la aplicación usa clave publicable y cookies SSR. auth.getUser verifica el usuario, actorFromUser toma permisos de app_metadata, nunca user_metadata. RLS y las RPC vuelven a comprobar permisos en base. No se utiliza service_role en el flujo web. La clave pública no permite aprovisionar usuarios ni ejecutar migraciones administrativas.

Next/React/TypeScript permiten compartir tipos, UI y API con herramientas de prueba ya instaladas. PostgreSQL aporta FK, CHECK, EXCLUDE y transacciones; Supabase ofrece Auth/PostgREST y RLS sin construir otro backend. El costo académico se reduce al reutilizar el stack y ejecutar QA en un contenedor local; no se validaron cuotas/precios del servicio ni se contrataron recursos. Escalabilidad depende de índices, paginación, límites de consultas y capacidad del proyecto, no solo del framework. Las políticas y migraciones versionadas favorecen mantenimiento y despliegues reproducibles.

GitHub Actions ejecuta npm ci y npm run verify. TAR05 añade un job PostgreSQL aislado con las pruebas SQL y concurrencia. El job se revisó localmente mediante su mismo script; no se afirma una corrida de Actions remota. El dominio Vercel está configurado en sitemap, pero no se verificó un despliegue. La infraestructura de recordatorios pg_cron/pg_net/Vault configura un job inactivo: no es un módulo de notificaciones completamente activo ni se probó en PostgreSQL local.

C. Matriz RF-01 a RF-06

RF-01 Registrar: src/features/appointments/service.ts, validation.ts, supabase-repository.ts; src/app/api/citas/route.ts y AppointmentForm.tsx. Alta válida, FK y duración se verifican en appointments.sql y tar05-integrity.sql. PASS SQL local y tests Jest. Navegador autenticado remoto: BLOQUEADO.

RF-02 Consultar: service.ts, supabase-repository.ts, AppointmentAgenda.tsx y AppointmentList.tsx. Lectura por permisos y datos relacionados: appointments.sql, appointment-actions.sql y Jest. PASS local. Playwright comprueba estado sin sesión y API 401. Persistencia en navegador autenticado remoto: BLOQUEADO.

RF-03 Reprogramar: API citas/[id], servicio, RPC reschedule_appointment y AppointmentActionDialog.tsx. Cambio válido, versión obsoleta, conflicto y cancelada: appointment-actions.sql. PASS SQL local y Jest. UI autenticada remota: BLOQUEADO.

RF-04 Cancelar: cancel_appointment, servicio y diálogo. Conservación de fila, eventos, reutilización del horario e idempotencia: appointment-actions.sql. PASS SQL local y Jest. No hay borrado físico como cancelación. UI autenticada remota: BLOQUEADO.

RF-05 Evitar traslapes: availability.ts, adaptador, EXCLUDE appointments_no_overlap y trigger de bloqueos. appointments.sql y ejecutor concurrente comprueban mismo médico rechazado, médicos distintos aceptados y dos sesiones con exactamente un ganador. PASS PostgreSQL local. Supabase remoto: BLOQUEADO.

RF-06 Confirmar: confirm_appointment y componentes de acciones. Confirmación persistida, cancelada rechazada y sin rol rechazado: appointment-actions.sql y null-role.sql. PASS SQL local y Jest. Recarga autenticada remota: BLOQUEADO.

Evidencias actuales por requisito: verificacion-bd-reproducible.log, pruebas-appointments.log, pruebas-actions.log, pruebas-integridad.log, regresion-rol-ausente.log y validacion-proyecto.log. Las pruebas de servicio/React usan dobles; las SQL se ejecutan en PostgreSQL real y no son reemplazables por esos dobles.

D. Modelo físico y Supabase

patients: PK UUID, nombre no vacío, indicador active y created_at timestamptz. physicians: mismas reglas para identidad mínima del médico. appointments: PK UUID, FK patient_id y physician_id con RESTRICT; starts_at/ends_at, duración 15/30/45/60, motivo, estados pending/confirmed/cancelled y metadatos de cancelación/confirmación. appointment_events: PK bigint identity, FK appointment_id RESTRICT, tipo y valores anteriores/nuevos del evento, actor y occurred_at. physician_schedule_blocks: PK UUID, FK physician_id RESTRICT, rango con zona horaria, motivo, created_at y EXCLUDE de bloques superpuestos.

Relaciones: patients 1:N appointments; physicians 1:N appointments; appointments 1:N appointment_events; physicians 1:N physician_schedule_blocks. Son cinco tablas de agenda sin expedientes, recetas, diagnósticos ni tratamientos. El bloqueo representa ausencias reales del médico, no una tabla artificial. Las entidades separan identidad, reserva, eventos y ausencia; los campos derivados de fin/historial son redundancia controlada por triggers, no una afirmación de normalización estricta de todos los atributos derivados.

Cada tabla tiene RLS. Admin/recepción consultan agenda y registran citas; médicos consultan su agenda y pacientes vinculados; cuentas sin rol no acceden. No hay UPDATE/DELETE directo de citas para authenticated. Las acciones se realizan por RPC con validación, bloqueo FOR UPDATE y versión esperada. Los eventos son append-only para usuarios normales. Bloqueos se administran con RLS por personal y son consultables por el médico correspondiente; no hay interfaz nueva para administrarlos en esta entrega.

Los intervalos son semiabiertos [inicio,fin), permiten citas contiguas. EXCLUDE protege reservas concurrentes. El trigger de bloqueos comparte bloqueo de la fila physicians entre creación de ausencia y reserva; el ejecutor prueba también esa carrera. Ninguna ausencia nueva invalida silenciosamente citas previas. No se usa ON DELETE CASCADE sobre historial.

Orden: 202610250001 infraestructura Supabase (no probada localmente), 202610250002 esquema canónico, 202610260001 comprobación de compatibilidad, 202610260003 acciones, 202611060001 reparación autorización, 202611060002 bloqueos. Se preservan los números históricos. La propuesta SQL descartada queda fuera de migrations.

Destino configurado: hvrkrycxbrnqpvyywpql.supabase.co. DNS ENOTFOUND al consultar Auth y REST. npx supabase projects list: AccessTokenRequiredError. No existe project-ref enlazado ni DATABASE_URL administrativa disponible en el entorno. No se pudo comprobar identidad académica del proyecto, tablas remotas, filas existentes, historial de migraciones ni RLS remoto. Migraciones aplicadas en Supabase durante TAR05: NINGUNA. Tablas remotas confirmadas: NINGUNA; esto significa no verificadas, no que estén vacías o

ausentes.

Pasos pendientes seguros: iniciar sesión CLI fuera del chat; confirmar el proyecto académico en el panel y su dominio; enlazar su ref con la sintaxis de la CLI instalada; consultar catálogo e historial usando consultas-remotas.sql; comparar esquema/filas e historial; ejecutar dry-run y revisar el plan; aplicar solo pendientes compatibles; verificar catálogo y repetir pruebas con cuentas ficticias normales. No usar migration repair para fingir ejecución ni cambiar versiones aplicadas sin inspección. Si existe el esquema alternativo o datos previos afectados, hace falta un plan de conversión con respaldo y confirmación específica antes de ejecutar DDL destructivo.

E. Pruebas y condiciones observadas

npm ci --no-audit --no-fund: PASS, 1061 paquetes instalados; avisos de dependencias obsoletas registrados en la sesión. npm run verify: PASS, ver validacion-proyecto.log. No se afirma un escaneo CVE completo de dependencias.

python scripts/tar05/verify-database.py: PASS. Crea un contenedor único sin red, aplica cinco migraciones de citas en transacciones, ejecuta cuatro suites SQL, prueba dos conexiones concurrentes, consulta pg_constraint/pg_tables y destruye únicamente su contenedor. Excluye la migración de infraestructura pg_cron/net/Vault por no ser parte de la imagen PostgreSQL estándar. auth-bootstrap.sql define roles y lectura de claims en esta base efímera; nunca debe aplicarse a Supabase.

PK duplicadas de las cinco tablas, FK de pacientes/médicos/eventos/bloqueos, estado inválido, duración cero/negativa, intervalos inválidos, solapamiento y ausencia: rechazo esperado y observado. Cita simultánea con médico diferente: aceptada en appointments.sql. Cancelación conserva fila e historial, libera horario y no duplica eventos: PASS appointment-actions.sql. Reserva concurrente: una confirma y otra es rechazada. Carrera bloqueo/reserva: exactamente un ganador. Datos sintéticos: transacciones ROLLBACK; escenarios concurrentes viven en contenedor efímero eliminado en finally.

RLS se prueba con SET LOCAL ROLE authenticated y claims de admin, médico y cuenta sin rol. La conexión inicial administrativa prepara fixtures; la autorización se evalúa bajo el rol SQL normal. Esto no verifica emisión de JWT, renovación de sesiones ni un inicio de sesión real de Supabase. null-role.sql prueba las tres RPC con rol ausente. El catálogo confirma RLS habilitado en las cinco tablas.

Playwright en app production local: escritorio 1440x1000, tablet 768x1024 y móvil 390x844. Se captura agenda sin sesión, acceso móvil, respuesta API 401, foco por teclado y prefers-reduced-motion. No hubo cambios visuales de diseño. Los errores de recursos 401 de la consola corresponden a las llamadas protegidas sin sesión; no se califican como ausencia total de errores de consola. No se instaló Axe porque el proyecto no lo tenía. Revisión de contraste completa y recorrido autenticado no certificados.

F. Auditoría crítica de IA y seguridad

El equipo declara uso de Codex y DeepSeek para planteamiento, elaboración, revisión, diseño y apoyo a pruebas. La asistencia no sustituye revisión humana, integridad ni evidencia reproducible. En esta auditoría se usó Codex; no se atribuyen cambios específicos previos a DeepSeek sin prueba de procedencia.

Problema 1: migración 202610260001 recreaba patients/physicians/appointments y cambiaba starts_at por scheduled_at. Reproducción: ERROR relation patients already exists después de ejecutar 202610250002. Riesgo: instalaciones nuevas fallan y el esquema alternativo no corresponde al adaptador. Corrección: archivar SQL original y conservar una comprobación no destructiva del contrato. Prueba: aplicación secuencial transaccional en contenedor nuevo, PASS. Fuente: commit 7963f97 integrado en e5299b5. Evidencia: reproduccion-migracion-original.log y verificacion-bd-reproducible.log.

Problema 2: IF appointment_role() NOT IN (...) no rechaza NULL. SECURITY DEFINER permitía avanzar sin rol, saltando la protección RLS del caller. Reproducción mínima: confirm_appointment con authenticated sin rol responde “La cita no existe” en lugar de 42501, demostrando paso por autorización. No fue necesario modificar una cita de otra persona para demostrarlo. Corrección aditiva: COALESCE(appointment_role(),'') en tres RPC. Prueba: null-role.sql exige insufficient_privilege para las tres. PASS. Fuente: 77bd031, migración 202610260003. Evidencia antes/después: reproduccion-rol-ausente.log y regresion-rol-ausente.log.

Problema 3: documentación decía que el borrador de Emilio no estaba fusionado y el quinto autor seguía pendiente. El grafo y e5299b5 demuestran lo contrario. Se corrigió docs/demostracion.md; el PDF PRY05 original se conserva intacto como antecedente. No se reconstruyó ni atribuyó falsamente ningún commit.

Limitación de atribución: no hay evidencia suficiente para afirmar que estos defectos fueron generados por una IA específica. Se documentan como deficiencias reales detectadas mediante auditoría asistida por IA, sin inventar procedencia. El diff y commit TAR05 identifican la corrección actual.

Escaneo acotado de archivos versionados por patrones de claves secretas/JWT: seguridad-secretos.json. Las variables privadas locales solo se inspeccionaron por nombre y presencia; no se imprimieron valores. No se alteraron claves, proxies, VNAI, DeepSeek ni configuración de otros agentes. La URL y clave publicable existentes se conservaron. No se certifica un pentest integral ni ausencia universal de secretos en todo el historial.

G. Archivos y reproducción

Migraciones: supabase/migrations/202611060001_authorization_null_role.sql y 202611060002_physician_schedule_blocks.sql; migración histórica 202610260001 corregida con archivo original en docs/sql-historico. Pruebas: supabase/tests/null-role.sql y tar05-integrity.sql más suites originales. Ejecutor: scripts/tar05/verify-database.py y auth-bootstrap.sql. CI: .github/workflows/verify.yml.

Evidencias: evidencias/TAR05/ contiene informe, historial con hashes/autores/fechas/archivos, ramas, estructura inicial, versiones, resultados SQL y de build, capturas Playwright reales y consultas remotas de solo lectura. PDF: evidencias/TAR05/TAR05-Amarillo.pdf. Generador del PDF: scripts/tar05/build-report.py. El PDF no contiene contraseñas ni firmas inventadas.

No se encontró TAR05-Amarillo.pdf previo en el directorio académico inspeccionado. Por tanto, esta es una nueva portada de reporte técnico; no se puede certificar conservación de una portada, declaración o firmas TAR05 que no estaban disponibles. PRY05-Equipo.pdf permanece intacto. Los únicos nombres usados como autores son los verificables en Git; no se completaron nombres legales ni firmas faltantes.

H. Pendientes y cierre

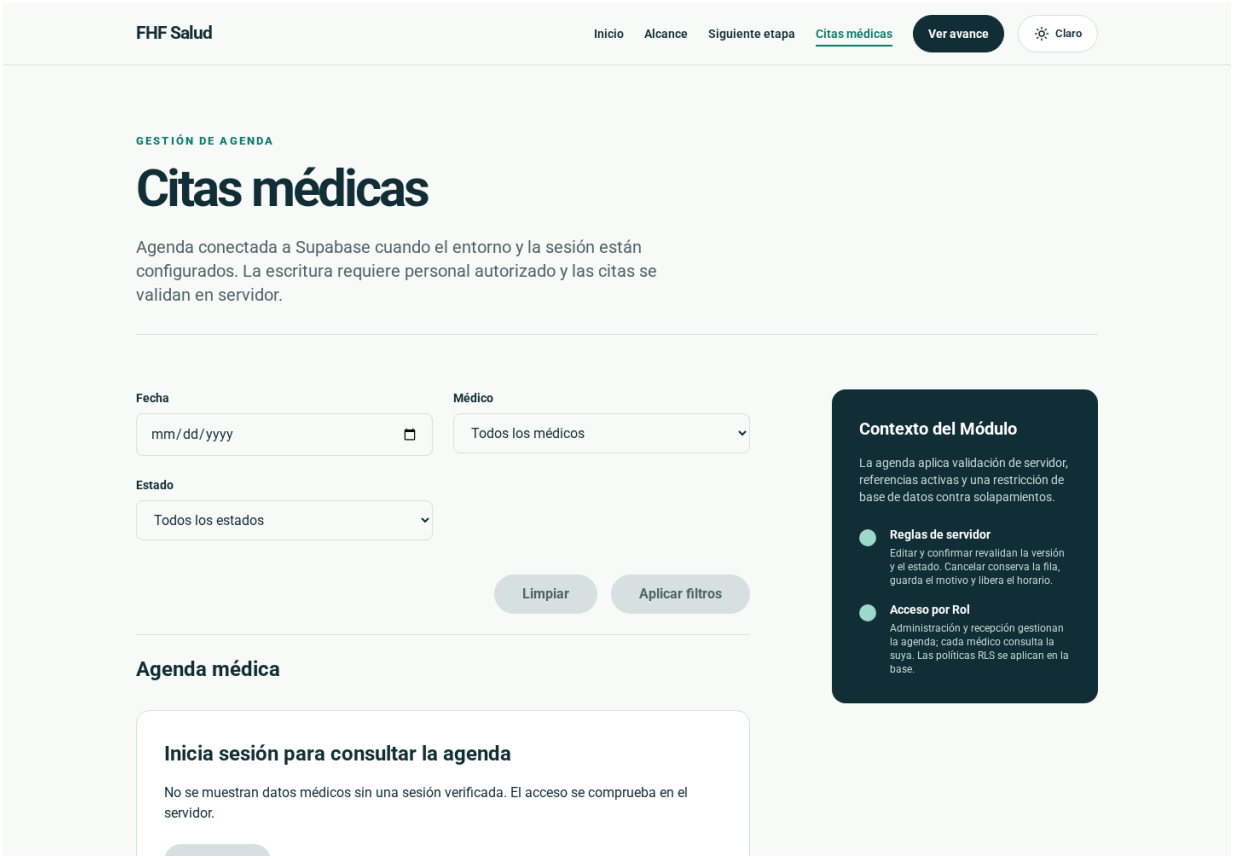
1. Recuperar acceso al proyecto Supabase académico correcto y resolver DNS ENOTFOUND. La clave publicable no autoriza DDL ni inspección administrativa. Se solicitó iniciar sesión CLI y confirmar destino, sin compartir secretos en el chat. 2. Inspeccionar datos/esquema/historial remotos antes de aplicar y verificar migraciones, constraints y RLS. Si hay datos afectados o conversión destructiva, obtener confirmación específica. Ejecutar Auth y RF-01 a RF-06 de navegador con cuentas normales y recarga contra Supabase real. 3. Incorporar portada/declaración/firmas TAR05 originales si existen fuera del entorno; no se generaron sustitutos de firmas. 4. Revisar y publicar la rama TAR05 mediante PR. No se publicó en main ni se enviaron mensajes a colaboradores. Los cambios locales anteriores siguen respaldados para reconciliación posterior.

Criterio de aceptación: NO COMPLETADO. Código y PostgreSQL local comprobados; falta aplicación y validación en Supabase remoto. Este reporte no transforma pruebas locales o evidencia PRY05 histórica en una certificación remota TAR05.

Anexo - capturas reales TAR05

Aplicación local en producción, sin sesión. Estas capturas no prueban un flujo autenticado ni conexión a la base remota.

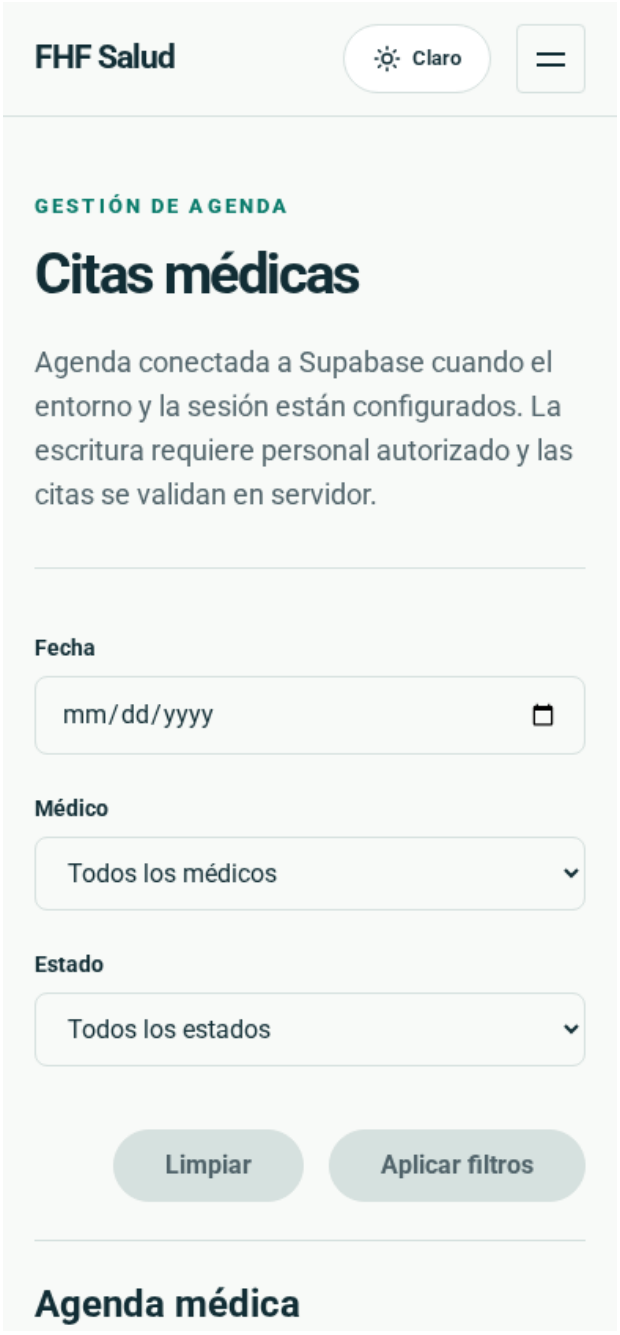
Agenda en escritorio (1440x1000).



Agenda en tablet (768x1024), movimiento reducido.



Agenda móvil (390x844), sin desbordamiento horizontal.



Formulario de acceso móvil, sin credenciales.

